Трансфер футболистов

1) Модель предметной области

На рисунке 1 представлена модель предметной области «Трансфер футболистов». Модель включает шесть классов и одно перечисление, их основные атрибуты и связи, отражающие структуру предметной области. Операции классов на диаграмме не приводятся.

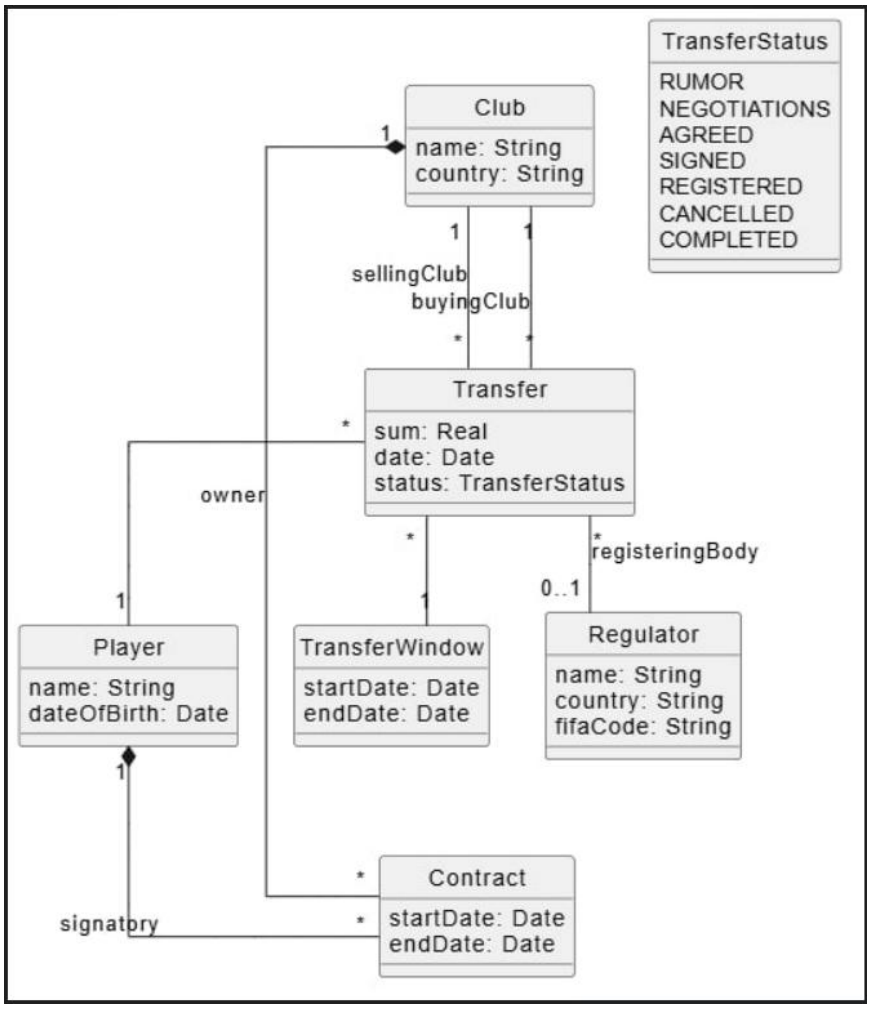


Рисунок 1 – Модель предметной области «Трансфер футболистов»

Клуб – футбольная организация, которая может выступать продающей или покупающей стороной трансфера и заключает контракты с игроками.

Игрок – футболист, являющийся предметом трансфера и стороной контракта.

Трансфер – переход игрока из одного клуба в другой с указанием суммы, даты и текущего статуса.

Трансферное окно – период, в течение которого разрешено оформление трансферов.

Контракт – соглашение между клубом и игроком, действующее в течение определённого срока.

Регулятор – организация, осуществляющая регистрацию трансферов.

Статус трансфера (TransferStatus) – перечисление, задающее этапы жизненного цикла трансфера: RUMOR, NEGOTIATIONS, AGREED, SIGNED, REGISTERED, CANCELLED, COMPLETED.

Связь «Клуб – Контракт» (роль owner) является композицией: контракт принадлежит клубу-владельцу, а клуб может иметь множество контрактов. Связь «Игрок – Контракт» (роль signatory) также является композицией: каждый контракт относится к одному игроку-подписанту, а игрок может иметь множество контрактов.

С трансфером клуб связан двумя ролями: sellingClub (продающий) и buyingClub (покупающий). У каждого трансфера ровно один продающий и один покупающий клуб, а клуб может участвовать во множестве трансферов в каждой из ролей. Каждый трансфер относится ровно к одному игроку и ровно к одному трансферному окну, при этом игрок и окно могут быть связаны со множеством трансферов. Роль registeringBody указывает на регулятора, зарегистрировавшего трансфер: у трансфера может не быть регулятора либо быть один, а один регулятор может регистрировать множество трансферов.

2) Диаграмма коммуникаций

На рисунке 2 показано взаимодействие участников при обработке заявки на трансфер. В сценарии участвуют актёр «Представитель покупающего клуба» и объекты form:TransferForm, controller:TransferController, window:TransferWindow, currentContract:Contract, sellingClub:Club, buyingClub:Club, regulator:Regulator, transfer:Transfer. Объекты sellingClub и buyingClub являются экземплярами одного класса Club, выступающими в разных ролях.

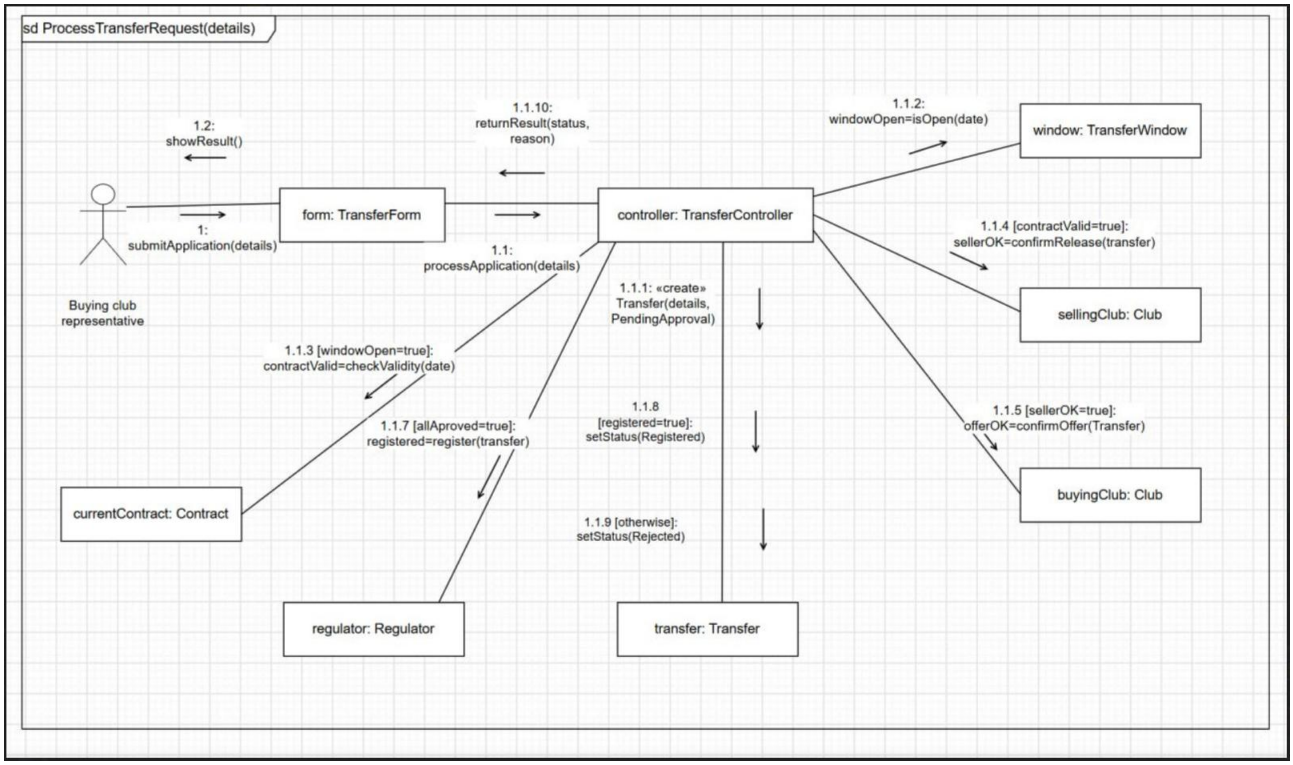


Рисунок 2 – Обработка заявки на трансфер

В данном сценарии участвуют девять участников: представитель покупающего клуба, форма заявки, контроллер трансфера, трансферное окно, текущий контракт, продающий клуб, покупающий клуб, регулятор и сам трансфер.

Представитель покупающего клуба подаёт заявку через форму (сообщение 1). Форма передаёт данные контроллеру (сообщение 1.1), который создаёт объект трансфера со статусом PendingApproval (сообщение 1.1.1).

Контроллер проверяет, открыто ли трансферное окно на указанную дату (сообщение 1.1.2). Если окно открыто, проверяется действительность текущего контракта игрока (сообщение 1.1.3). При действующем контракте продающий клуб подтверждает отпуск игрока (сообщение 1.1.4), а при его согласии покупающий клуб подтверждает предложение (сообщение 1.1.5).

Если все подтверждения получены (allApproved=true), контроллер обращается к регулятору для регистрации трансфера (сообщение 1.1.7).

При успешной регистрации трансферу устанавливается статус Registered (сообщение 1.1.8), в противном случае – статус Rejected (сообщение 1.1.9). Сообщения 1.1.3–1.1.5 и 1.1.7–1.1.9 выполняются только при соблюдении указанных в них условий.

Результат (статус и причина) возвращается форме сообщением returnResult(status, reason) (сообщение 1.1.10), после чего форма показывает его представителю клуба (сообщение 1.2). Таким образом, диаграмма коммуникаций отражает последовательность проверок и подтверждений при оформлении трансфера.

1. **Взаимодействие Пользователя (Actor → Boundary):** Представитель покупающего клуба (`Buying club representative`) взаимодействует исключительно с экранной формой `TransferForm`, отправляя запрос `1: submitApplication(details)`. Обратно форма возвращает результат через `1.2: showResult()`.
2. **Делегирование Контроллеру (Boundary → Control):** Форма не содержит бизнес-логики и сразу перенаправляет вызов контроллеру `TransferController` с помощью сообщения `1.1: processApplication(details)`. По завершении процесса контроллер возвращает статус операции обратно в форму через `1.1.10: returnResult(status, reason)`.
3. **Управление Сущностями (Control → Entities):** Контроллер последовательно выполняет проверки и манипуляции над объектами данных:
    - Создает объект `transfer: Transfer` (`1.1.1: «create»`).
    - Проверяет статус трансферного окна в `window: TransferWindow` (`1.1.2: isOpen`).
    - Проверяет валидность текущего контракта игрока в `currentContract: Contract` (`1.1.3: checkValidity`).
    - Запрашивает подтверждение отпуска игрока у `sellingClub: Club` (`1.1.4: confirmRelease`).
    - Запрашивает подтверждение оффера у `buyingClub: Club` (`1.1.5: confirmOffer`).
    - Регистрирует трансфер в регулирующем органе `regulator: Regulator` (`1.1.7: register`).
    - В зависимости от прохождения всех проверок, обновляет состояние сущности `transfer` (`1.1.8: setStatus(Registered)` либо `1.1.9: setStatus(Rejected)`).

3) Диаграмма анализа

На рисунке 3 представлена диаграмма анализа того же сценария с выделением ответственностей участников. В ней использованы роли Actor (представитель покупающего клуба), Boundary (форма заявки и экран результата), Control (контроллер трансфера) и пять сущностей Entity – трансферное окно, контракт, клубы и игрок, трансфер и регулятор.

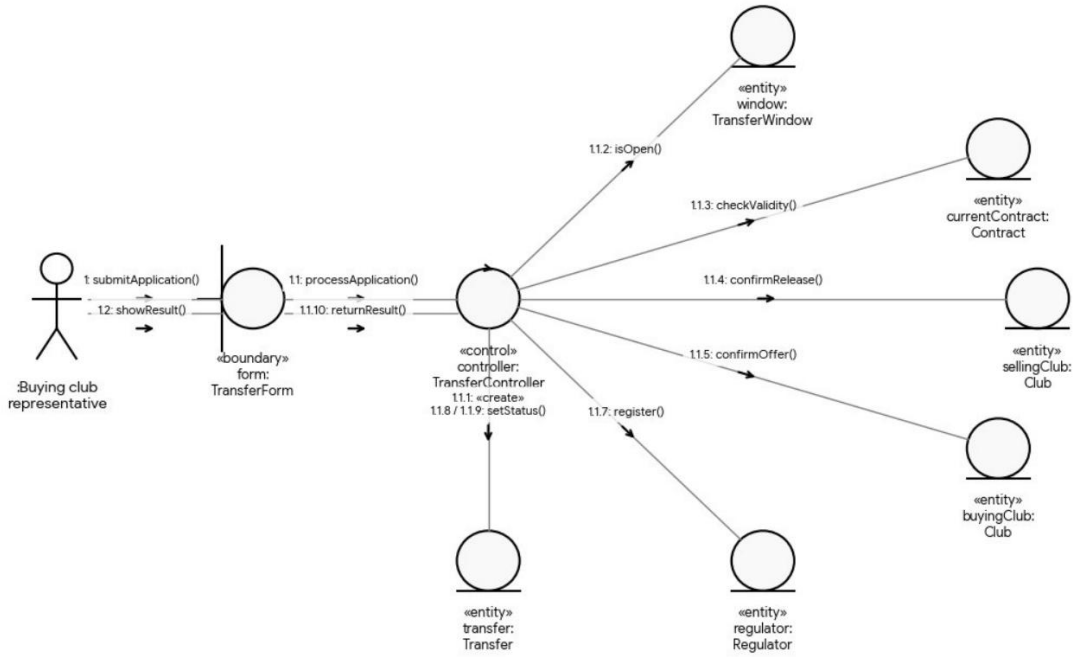


Рисунок 3 – Диаграмма анализа: оформление трансфера

1. **Взаимодействие Пользователя (Actor → Boundary):** Представитель покупающего клуба (`Buying club representative`) взаимодействует исключительно с экранной формой `TransferForm`, отправляя запрос `1: submitApplication(details)`. Обратно форма возвращает результат через `1.2: showResult()`.
2. **Делегирование Контроллеру (Boundary → Control):** Форма не содержит бизнес-логики и сразу перенаправляет вызов контроллеру `TransferController` с помощью сообщения `1.1: processApplication(details)`. По завершении процесса контроллер возвращает статус операции обратно в форму через `1.1.10: returnResult(status, reason)`.
3. **Управление Сущностями (Control → Entities):** Контроллер последовательно выполняет проверки и манипуляции над объектами данных:
- Создает объект `transfer: Transfer` (`1.1.1: «create»`).
- Проверяет статус трансферного окна в `window: TransferWindow` (`1.1.2: isOpen`).
- Проверяет валидность текущего контракта игрока в `currentContract: Contract` (`1.1.3: checkValidity`).
- Запрашивает подтверждение отпуска игрока у `sellingClub: Club` (`1.1.4: confirmRelease`).
- Запрашивает подтверждение оффера у `buyingClub: Club` (`1.1.5: confirmOffer`).
- Регистрирует трансфер в регулирующем органе `regulator: Regulator` (`1.1.7: register`).
- В зависимости от прохождения всех проверок, обновляет состояние сущности `transfer` (`1.1.8: setStatus(Registered)` либо `1.1.9: setStatus(Rejected)`).